Практическая работа 2.

Курсоры.

В языке SQL курсоры служат указателями, которые позволяют языку написания приложений обрабатывать результаты запросов построчно. Данные в реляционной базе данных рассматриваются как множества. Как следствие, результаты запроса возвращаются операторами SELECT в виде результирующих наборов. Эти результирующие наборы есть не что иное, как одна или более строк и столбцов, извлеченных из одной или более таблиц. Вы можете пролистать результирующие наборы, чтобы вытащить необходимую вам информацию. Модель базы данных SQL имеет три основных структуры:

- столбцы (или атрибуты) и их типы данных;
- строки (записи или кортежи);
- таблицы (коллекция записей).

Результат, возвращаемый операторами SELECT, является мультимножеством записей, где каждая запись является коллекцией атрибутов. Базовые языки программирования обычно работают с отдельными значениями данных кортежа, возвращаемого запросом. Таким образом, является существенным, чтобы результат запроса SQL отображался на структуры данных, поддерживаемых языком программирования. Механизм обхода кортежей в цикле необходим для организации итераций по кортежам и значениям их атрибутов.

Курсор действует подобно переменной итератора, чтобы обойти кортежи, возвращаемые запросом SQL, и извлечь отдельные значения в пределах каждого кортежа, которые затем могут отображаться на соответствующие типы переменных базового языка программы.

Результат выполнения запроса может содержать много строк, но базовый язык приложения может обработать одновременно только одну строку. В результате, курсор объявляется как встроенный в соответствующий язык программирования оператор SQL. Затем курсор открывается подобно файлу, и извлекается отдельная строка из результата запроса. Остальные строки извлекаются последовательно до тех пор, пока курсор не будет закрыт.

Объявление курсора:

Курсоры объявляются во многом подобно переменной. Курсору дается имя, имеются операторы для открытия курсора, извлечения результатов запроса и, наконец, закрытия курсора. Отметим, что различные реализации SQL поддерживают

использование курсоров по-разному. Но здесь имеется общее соглашение как должен записываться курсор.

Мы должны использовать несколько операторов SQL для полной реализации функциональности курсора, т.к. одно лишь объявление курсора недостаточно для извлечения данных из базы данных. Вот 4 основных шага для работы с курсором:

DECLARE CURSOR: Объявление начинается с задания имени курсору и присвоения ему выражения запроса, которое будет вызываться при открытии курсора.

OPEN: Оператор open выполняет выражение запроса курсора и подготавливает результат запроса для последующего обхода (FETCH).

FETCH: Извлекает значения данных в переменные, которые затем могут передаваться базовому языку программирования или другим встроенным операторам SQL.

CLOSE: Курсор закрывается, и больше не может извлекать результаты запроса.

Курсоры в MySQL

В MySQL имеется два основных типа курсоров: только на чтение (read-only) и только вперед (forward-only). Эти курсоры могут использоваться в хранимых процедурах MySQL. Они помогают нам итерационно обходить результаты запроса по одной строке за раз и извлекать значения в переменные для последующей обработки. Имеется возможность объявлять более одного курсора и вкладывать их в циклы. Заметим, что курсоры являются курсорами только на чтение, поскольку они для итераций используют временные таблицы. Обычно курсор выполняет запрос при его открытии.

Одной из проблем, связанных с курсорами в MySQL, является их отрицательное влияние на производительность из-за дополнительных операций ввода/вывода, которые они производят. Это особенно справедливо для больших типов данных, таких как BLOB и TEXT. Т.к. курсоры работают с временными таблицами, эти типы не поддерживаются в таблицах в памяти (in-memory). Следовательно, при работе с этими типами данных MySQL должна создавать временные таблицы на диске, что требует множества операций ввода/вывода. Это основная причина низкой производительности курсоров.

MySQL также не поддерживает курсоров на стороне клиента, однако клиентское API может эмулировать их при необходимости. Но тогда это не сильно отличается от извлечения результата в массив на языке программирования типа Java, и манипулирования этим массивом вместо использования курсора.

Пример.

Для корректной работы процедуры можно сформировать курсор. Для основы была взята ранее разработанная процедура с входными и выходными данными. Выходные

данные в этом случае будут записаны в курсор для их лучшей обработки, входные данные останутся неизменными.

```
DELIMITER |
create procedure test_curr(in city varchar(100))
begin
 DECLARE done INT DEFAULT FALSE;
    DECLARE fn varchar(14);
    DECLARE count_id int(3);
     DECLARE mycursor CURSOR FOR
SELECT passengers.name_pass, count(register.id_avia)
from passengers INNER JOIN register on passengers.id_pass=register.id_passenger
inner join avia on avia.id_avia=register.id_avia
inner join city on avia.id_city=city.id_city
WHERE city.name_city=city;
 DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
    OPEN mycursor;
  fetch_loop: LOOP
    FETCH mycursor INTO fn, count_id;
  IF done THEN
      LEAVE fetch_loop;
    END IF;
    SELECT fn, count_id;
  END LOOP;
  CLOSE mycursor;
  END |
  DELIMITER ;
```

Рисунок 1 – Создание курсора внутри хранимой процедуры

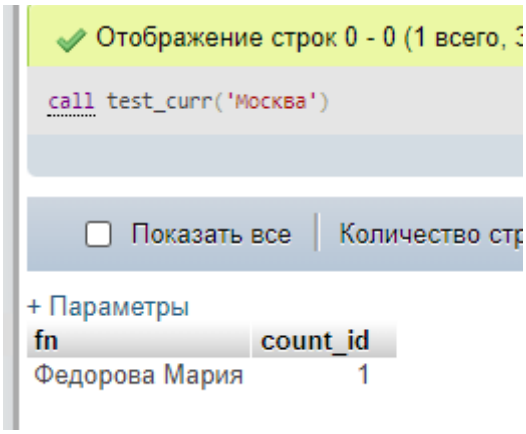


Рисунок 2 – Результат работы хранимой процедуры

Из незнакомых строк здесь встречается DECLARE CONTINUE HANDLER FOR NOT FOUND. Таким образом обозначен синтаксис установки обработчика для условия NOT FOUND для курсора в MySQL, за то, что курсор найден отвечает логическая переменная done, принимающая значение true.

Запустив аналогичную процедуру с входными и выходными данными можно убедиться, что результат не изменился.

Задание.

Модифицировать имеющиеся две хранимые процедуры, создав внутри них курсоры. Проверить корректность работы процедур с курсорами.

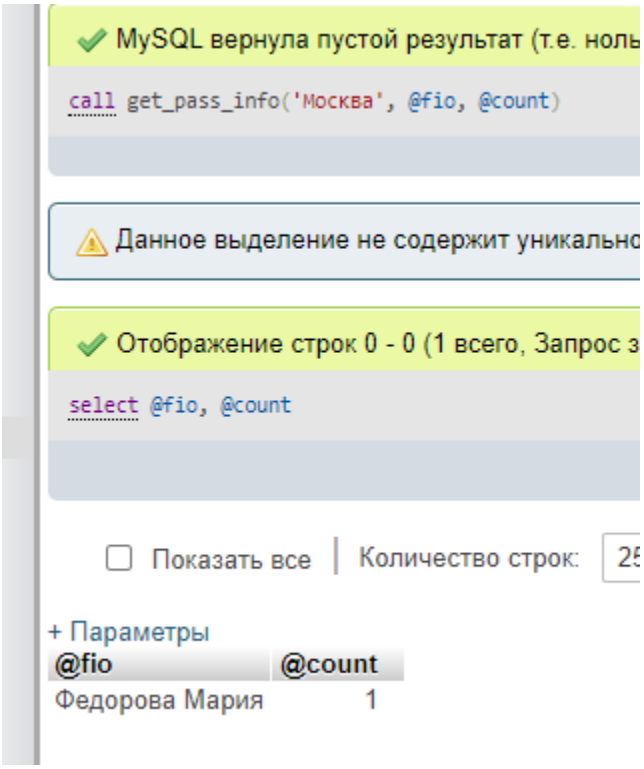


Рисунок 3 – Результат работы хранимой процедуры до модификации